

COMMAND PROTOCOL DESIGN - IA_AUTOMATE

Arquitetura de Comunicação entre Agente de IA e Sistema de Automação Multimodal

06 de maio de 2026

1. PROBLEMA

O desenvolvimento de agentes autônomos enfrenta um gargalo crítico na tradução de intenções cognitivas em ações executáveis no sistema operacional. Atualmente, a Inteligência Artificial gera **pseudocódigo**— uma sequência de instruções em linguagem natural que, embora compreensível para humanos, carece de determinismo para o sistema de automação. O sistema receptor precisa capturar, validar e executar esses comandos sem ambiguidade. A ausência de uma **linguagem formal estruturada** entre a IA e o Sistema resulta em falhas de execução, dificuldade de depuração e riscos de segurança, uma vez que o sistema não consegue prever o impacto de uma instrução mal formatada antes de sua invocação.

2. REQUISITOS DO PROTOCOLO

Para garantir a robustez da interface entre o agente e o hardware/software do PC, o protocolo de comando deve atender aos seguintes critérios técnicos:

- Facilidade de Parsing:** A estrutura deve ser processável por algoritmos padrão (JSON, XML ou DSL customizada) com baixo custo computacional.
- Validação Prévia:** O protocolo deve permitir a verificação de sintaxe e permissões antes que qualquer alteração seja persistida no disco ou executada na memória.
- Extensibilidade de Ações:** Suporte nativo para operações fundamentais como **create_file**, **modify_file**, **run_script**, **screenshot**, **browser_control** e **audio_output**.
- Contextualização:** Cada comando deve carregar metadados sobre o ambiente alvo, incluindo caminhos de arquivos, escopo de funções e parâmetros de configuração.
- Loop de Feedback:** O protocolo deve prever campos de retorno para que o sistema reporte sucesso, erro (stack trace) ou resultados de saída (logs) de volta para a IA.

Atenção: A falta de um esquema de validação rigoroso pode permitir a execução de comandos destrutivos no sistema de arquivos do host.

3. PROPOSTA 1 - JSON FORMAT

Esta proposta utiliza a estrutura **JSON (JavaScript Object Notation)**, padrão de mercado para APIs, garantindo compatibilidade total com a maioria das linguagens de programação e bibliotecas de validação de esquema (JSON Schema).

3.1 Estrutura de Dados

- **comando**: String definindo a ação (ex: edit_file).
- **parametros**: Objeto contendo os dados necessários para a execução.
- **contexto**: Metadados sobre o projeto e permissões necessárias.
- **segurança**: Campo para **checksum** (SHA-256) do conteúdo e tokens de autorização.

3.2 Exemplos de Implementação

Exemplo 1: Criar arquivo Python

```
{  
  
  "comando": "create_file",<br/>  
  
  "parametros": {<br/>  
  
    "nome": "app_monitor.py",<br/>  
  
    "tipo": "python",<br/>  
  
    "conteudo": "import os  
  
print(os.getlogin())"  
  
  },  
  
  "contexto": { "path": "/src/modules", "overwrite": true },<br/>  
  
  "segurança": { "permissions": "755" }  
  
}
```

Exemplo 2: Editar linhas específicas de um arquivo

```
{  
  
  "comando": "edit_file",<br/>
```

```
"parametros": {<br/>
"arquivo": "config.json",<br/>
"operacao": "replace",<br/>
"range": { "start": 10, "end": 12 },<br/>
"novo_conteudo": " \\"debug\":" true,"
},
"contexto": { "backup_before": true }
}
```

Exemplo 3: Executar teste

```
{
"comando": "run_script",<br/>
"parametros": {<br/>
"engine": "pytest",<br/>
"target": "tests/test_core.py",<br/>
"args": ["-v", "--maxfail=1"]
},
"contexto": { "timeout": 30 }
}
```

4. PROPOSTA 2 - DSL (DOMAIN SPECIFIC LANGUAGE)

Uma **Linguagem de Domínio Específico** customizada, otimizada para brevidade e legibilidade, utilizando prefixos de controle para facilitar a captura via Regex pelo sistema de monitoramento do clipboard ou stream de texto.

4.1 Sintaxe Proposta

- Prefixo de comando: @#\$-

- Atributos: chave=valor
- Conteúdo: Delimitado por aspas triplas ou blocos específicos.

4.2 Exemplos de Implementação

Exemplo 1: Criar arquivo Python@#\$_create-file arquivo="app_monitor.py"
tipo=python conteudo="""import os print(os.getlogin())"""

Exemplo 2: Editar linhas específicas de um arquivo@#\$_edit-file
arquivo="config.json" linhas=10-12 conteudo=""" "debug":
true,"""

Exemplo 3: Executar teste@#\$_run-script engine="pytest"
target="tests/test_core.py" args="--v --maxfail=1"

5. PROPOSTA 3 - HYBRID (PSEUDOCÓDIGO + METADADOS)

O formato **Híbrido** busca equilibrar a natureza discursiva das LLMs com a necessidade de tags estruturadas. Ele permite que a IA explique o que está fazendo antes de emitir o bloco de comando técnico.

5.1 Estrutura de Tags

- Blocos delimitados por colchetes [TAG] para metadados.
- Separadores claros --- para o corpo do conteúdo.

5.2 Exemplos de Implementação

Exemplo 1: Criar arquivo Python

[COMANDO: create_file]

[ARQUIVO: app_monitor.py]

[TIPO: python]

import os

print(os.getlogin())

Exemplo 2: Editar linhas específicas de um arquivo

[COMANDO: edit_file]

[ARQUIVO: config.json]

[LINHAS: 10-12]

"debug": true,

Exemplo 3: Executar teste

[COMANDO: run_script]

[ENGINE: pytest]

[TARGET: tests/test_core.py]

[ARGS: -v, --maxfail=1]

6. ANÁLISE COMPARATIVA

Abaixo, apresenta-se a comparação técnica entre os protocolos propostos para fundamentar a decisão arquitetural:

Critério	JSON Estruturado	DSL Customizada	Hybrid (Tags)
Facilidade de Parse	Altíssima (Nativo)	Média (Requer Regex)	Média (Parser de Bloco)
Legibilidade Humana	Média	Alta	Altíssima
Flexibilidade	Alta	Baixa	Média
Segurança	Alta (Schema Validation)	Média	Baixa
Overhead	Baixo	Mínimo	Médio

7. RECOMENDAÇÃO

Para o projeto **IA_Automate**, a recomendação técnica é a adoção da **Proposta 1 (JSON Estruturado)** como o protocolo de transporte principal, porém encapsulado em uma interface de saída que aceite a **Proposta 3 (Hybrid)** para facilitar a geração pela IA.

Justificativa:

1. **Robustez:** O JSON permite o uso de *JSON Schema* para validar se todos os parâmetros obrigatórios estão presentes antes de qualquer execução, reduzindo drasticamente erros de runtime.
2. **Integração:** Como o projeto já utiliza scripts PowerShell e manipulação de arquivos via JSON (bootstrap/restore), manter a consistência de dados simplifica a arquitetura.
3. **Segurança:** A facilidade de incluir campos de *checksum* assinaturas digitais no JSON garante que o comando recebido pelo sistema é exatamente o que a IA pretendia enviar, sem corrupção de dados no clipboard.

"A padronização via JSON transforma a IA de um gerador de texto em um verdadeiro driver de sistema, permitindo auditoria e controle granular sobre cada ação executada no ambiente host."

Documento elaborado em 06 de maio de 2026. As informações contidas são de responsabilidade do solicitante.